

Architetture e Sistemi Operativi

Enrico Melis

a.a. 2025-2026

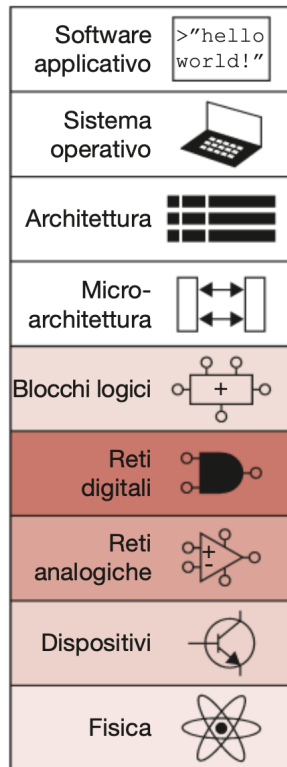
Indice

1	Introduzione al corso	3
1.1	Livelli di astrazione	3
1.2	Nozioni sui transistor	4
1.3	Logica booleana	4
2	Reti Logiche	5
2.1	Reti logiche combinatorie	6
2.2	Ottimizzazione nella logica combinatoria	8
2.3	Verilog	8
2.4	Reti logiche sequenziali	8
2.5	Componenti standard	8
3	Memorie	8
4	Assembler	8
4.1	Istruzioni standard e utilizzo	8
4.2	Codifica della istruzioni	8
5	Parallelismo	8
6	Microarchitetture	8
6.1	Single cycle	8
6.2	Multi cycle	8
6.3	Pipeline	8

1 Introduzione al corso

1.1 Livelli di astrazione

Come possiamo vedere anche nel libro [1], arriviamo ai sistemi operativi partendo da concetti molti più bassi.



Tralasciando la fisica e accennando la parte sui transistor, iniziamo il nostro cammino di astrazione con la reti logiche. I livelli di astrazione trattati sono (con alcuni esempi):

- Sistema Operativo (driver dei dispositivi)
- Architettura (istruzioni assembler, registri)
- Microarchitettura (datapath)
- Blocchi logici (sommatori, memorie)
- Reti digitali (gate logici AND, OR)

I livelli di astrazioni sono fondamentali per comprendere come funzionano tutte le componenti che diamo per scontato, costruendole man mano dal basso e incrementando la complessità.

1.2 Nozioni sui transistor

(In italiano vengono chiamati "transistor", ma mi rifiuto). I transistor sono, al giorno d'oggi, l'unità di misura principale per gli effettivi calcoli all'interno dei calcolatori. I primi calcolatori sono stati costruiti con degli ingranaggi, adesso si utilizzano i transistor che si dividono in:

- BJT: *Bipolar Junction Transistors*
- MOS: *Metal-Oxide-Semiconductor Field Effect Transistor (MOSFET)*

I transistor MOS sono costruiti partendo dal silicio (Si, IV gruppo), che forma 4 legami covalenti. Per aumentare la conduttività vengono aggiunti degli atomi droganti, che possono essere del V gruppo (arsenico) o del III (boro). Dato che l'arseno "ha un elettrone in più" e porta una carica negativa, viene chiamato drogante di tipo **n**, mentre il boro "porta una carica positiva"¹ e quindi viene chiamato drogante di tipo **p**.

**** Work in progress ****

1.3 Logica booleana

La logica booleana² è quella sottosezione dell'algebra in cui le variabili possono avere solamente valori binari, cioè $x \in \{0, 1\}$ oppure in senso astratto *vero* o *falso*. Le basi si fondano sui gate logici AND($\wedge, \cdot$), OR($\vee, +$), NOT($\neg, \bar{x}$) che sono realizzabili tramite transistor.

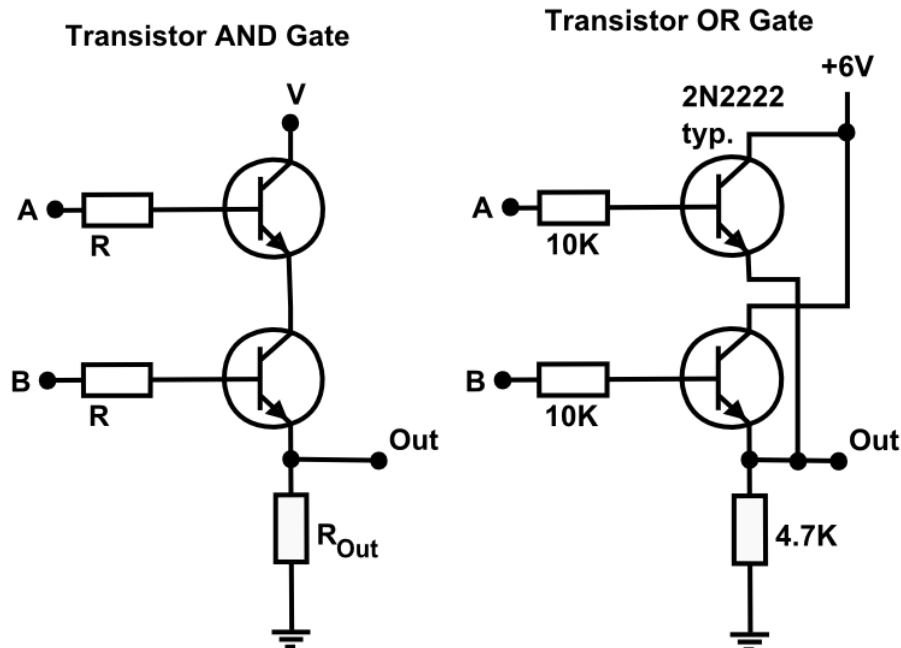
Queste sono le tabelle di verità per i gate logici appena menzionati:

p	q	$p \cdot q$	p	q	$p + q$	p	$\bar{p}$
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

Per avere una rappresentazione visiva di come sono realizzabili tramite transistor, ci basti pensare che per ottenere un AND basta mettere due transistor in sequenza, per l'OR basta metterli in parallelo e per il NOT basta invertire il segnale in input. Di seguito delle illustrazioni prese da Wikipedia.

¹Spero che chimica, fisici e ingegneri elettronici mi risparmieranno per questo scempio che ho appena scritto però rende l'idea.

²George Boole (1815 – 1864) è stato un matematico e logico inglese.



1.3.1 Composizionalità

È importante tenere a mente che questi tre gate logici sono i veri gate fondamentali. Spesso si possono trovare altri gate categorizzati in questo modo ma lo sono soltanto per astrazione, dato che tutti i circuiti logici sono costruibili tramite questi 3. Ad esempio, il gate XOR è spesso considerato tra i fondamentali però non è altro che una composizione in formula dei precedenti tre già citati. La formula logica per lo XOR infatti è $A \oplus B = ((A \wedge \neg B) \vee (\neg A \wedge B))$. Capire questo è fondamentale sia per capire i livelli di astrazione (l'esempio dello XOR serve a capire quanto sia utilizzato e quanto possa essere utile astrarre la sua composizione per comodità) e il concetto di ritardo dei segnali. Ogni gate logico ha bisogno di un certo tempo Δt per stabilizzarsi e in base a come componiamo i vari gate questi tempi si sommano. La composizionalità è fondamentale anche per capire il concetto di *modularità*, fondamentale nella filosofia UNIX.

2 Reti Logiche

Le reti logiche sono dei circuiti che elaborano segnali logici booleani tramite la composizione dei gate logici fondamentali. Possono essere usate per calcolare funzioni o per memorizzare segnali.

2.1 Reti logiche combinatorie

Le reti logiche combinatorie sono quei circuiti logici che permettono di modellare delle relazioni di input-output. La procedura standard per creare una rete logica combinatoria è:

1. Descrizione della funzione
2. Tabella di verità
3. Formula (somma di prodotti)
4. Rete di porte

dove con "somma di prodotti" si intende la scrittura della tabella di verità sotto forma di AND e OR, che sono rappresentabili rispettivamente come prodotti e somme.

Primo esempio di rete combinatoria: Full Adder a 1 bit Vogliamo creare una rete che permetta di sommare due bit, producendo un riporto.

- Input: bit x , bit y , riporto in ingresso
- Output: bit z , riporto in uscita

Descriviamo la funzione. Vogliamo una funzione che faccia la somma di due bit, prendendo in ingresso un bit di riporto e che sia in grado di produrre un bit di riporto in uscita (per composizionalità).

Creiamo la tabella di verità.

x	y	c_{in}	z	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Scriviamo la formula come somma di prodotti. Per farlo, individuiamo nella tabella di verità tutte le righe in cui l'uscita vale 1, separatamente per z e per c_{out} .

Nel caso di z , l'uscita vale 1 nella seconda, terza, quinta e ottava riga. Da ciascuna di queste righe costruiamo un *prodotto* (AND) tra x , y e c_{in} che risulta vero solo per quella combinazione di ingressi: se in una riga un ingresso vale 0, usiamo la variabile negata; se vale 1, usiamo la variabile non negata. Infine facciamo l'OR (somma) di tutti questi prodotti, così che z valga 1 ogni volta che si verifica almeno una delle righe selezionate.

$$z = \bar{x} \cdot \bar{y} \cdot c_{\text{in}} + \bar{x} \cdot y \cdot \overline{c_{\text{in}}} + x \cdot \bar{y} \cdot \overline{c_{\text{in}}} + x \cdot y \cdot c_{\text{in}}$$

In questo modo, la formula di z rappresenta esattamente tutte le combinazioni di input per cui $z = 1$. Facciamo la stessa cosa per c_{out} , sommando (in OR) i prodotti costruiti a partire dalle righe in cui $c_{\text{out}} = 1$:

$$c_{\text{out}} = x \cdot y \cdot \overline{c_{\text{in}}} + x \cdot \bar{y} \cdot c_{\text{in}} + \bar{x} \cdot y \cdot c_{\text{in}} + x \cdot y \cdot c_{\text{in}}$$

2.2 Ottimizzazione nella logica combinatoria

2.3 Verilog

2.4 Reti logiche sequenziali

2.4.1 Automi di Mealy

2.4.2 Automi di Moore

2.5 Componenti standard

2.5.1 Full Adder

2.5.2 Mux e Demux

2.5.3 Comparatore

2.5.4 Shifter

2.5.5 Encoder e Decoder

3 Memorie

4 Assembler

4.1 Istruzioni standard e utilizzo

4.2 Codifica della istruzioni

5 Parallelismo

6 Microarchitetture

6.1 Single cycle

6.2 Multi cycle

6.3 Pipeline

6.3.1 Dipendenze

6.3.2 Valutazione dei tempi di esecuzione

Riferimenti bibliografici

- [1] Sarah L. Harris David Money Harris. *Sistemi digitali e architettura dei calcolatori*. Zanichelli, 2017.